

```

"Active"   : true,
"Price"    : 30.66,
"Shares outstanding" : 8.38e9,
"Related companies" :
  [ "HPQ", "IBM", "YHOO", "DELL", "GOOG" ]
}

```

A *value* in JSON can be one of several types. An *object* in JSON is a comma-separated sequence of key-value pairs, wrapped in curly braces (`{}`). The keys must be strings like "Ticker" or "Price", and the values can be either another object, an *array* like `["HPQ", "IBM" ...]` that contains further values, or a *literal* like "MSFT", `true`, `null`, or `30.66`.

We'll write a rather dumb parser that simply parses a syntax tree from the document without doing any further processing.¹¹ We'll need a representation for a parsed JSON document. Let's introduce a data type for this:

```

trait JSON
object JSON {
  case object JNull extends JSON
  case class JNumber(get: Double) extends JSON
  case class JString(get: String) extends JSON
  case class JBool(get: Boolean) extends JSON
  case class JArray(get: IndexedSeq[JSON]) extends JSON
  case class JObject(get: Map[String, JSON]) extends JSON
}

```

9.4.2 A JSON parser

Recall that we've built up the following set of primitives:

- `string(s)`: Recognizes and returns a single `String`
- `regex(s)`: Recognizes a regular expression `s`
- `slice(p)`: Returns the portion of input inspected by `p` if successful
- `succeed(a)`: Always succeeds with the value `a`
- `flatMap(p) (f)`: Runs a parser, then uses its result to select a second parser to run in sequence
- `or(p1, p2)`: Chooses between two parsers, first attempting `p1`, and then `p2` if `p1` fails

We used these primitives to define a number of combinators like `map`, `map2`, `many`, and `many1`.



EXERCISE 9.9

Hard: At this point, *you* are going to take over the process. You'll be creating a `Parser[JSON]` from scratch using the primitives we've defined. You don't need to

¹¹ See the chapter notes for discussion of alternate approaches.